

Chapter 4

3D and Video

In our journey through computer vision this far, we have focused exclusively on how neural networks process static 2D images. However, the real world is intrinsically three-dimensional and relentlessly dynamic; objects possess depth and volume, and scenes evolve over time. Capturing this spatial and temporal richness is absolutely essential for advanced tasks like 3D reconstruction, object tracking, and video action analysis.

This chapter serves as a crucial bridge between flat 2D perception and a comprehensive understanding of space and time. We will first examine how deep learning adapts to 3D data, exploring foundational representations like depth maps, surface normals, voxels, and point clouds, alongside the specialized loss functions—such as scale-invariant loss and cosine distance—required to predict them. Next, we will transition into the temporal domain to analyze video, surveying architectural strategies ranging from Single-Frame CNNs and fusion techniques to 3D Convolutions and Two-Stream networks for action recognition. Finally, we will review the persistent challenges in these domains, including high computational costs and the scarcity of labeled data, laying the groundwork for building AI systems that can seamlessly interact with complex, dynamic environments.

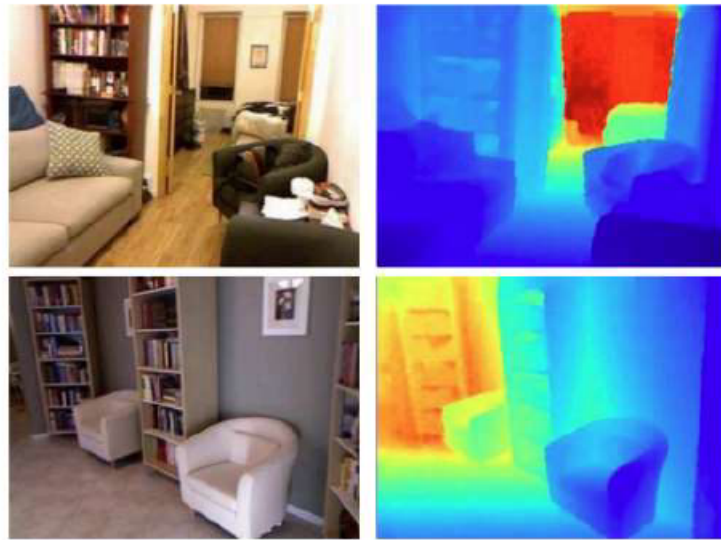
4.1 3D Representations

4.1.1 Depth Map

A **depth map** is a 2D representation in which each pixel stores information about the distance between the camera and objects in the scene in the real world. Instead of encoding color or light intensity, as in an RGB image, a depth map encodes depth: **small values represent points close to the camera, and large values indicate points farther away**. Visually, depth maps are usually represented in grayscale: white colors mean closer points and darker colors mean more distant points.

This representation is key to many computer vision and 3D graphics tasks, such as object pose estimation, and autonomous navigation. Figure 4.1 shows an example of two images and its depth maps. Note that in this case the depth map is no in grayscale, but a heatmap.

In any case, there is only one value (one channel) for each pixel.



RGB Image: $3 \times H \times W$ Depth Map: $H \times W$

Figure 4.1: Example of two images and its depth maps.

4.1.2 Surface Normals

A **surface normal** is a vector that is perpendicular to a surface at a given point. In computer vision and 3D graphics, a **normal map** is a 2D representation where each pixel stores the **orientation of the surface** at that point, encoded as a 3D vector (x, y, z) .



RGB Image: $3 \times H \times W$ Normals: $3 \times H \times W$

Figure 4.2: Example of two images and its normal maps.

Unlike a depth map, which indicates how far a point is from the camera, a normal map describes how the surface is oriented in space. This is essential for capturing details of the local geometry of objects or simulating how light interacts with surfaces (shading and rendering).

Visually, normal maps are usually represented by assigning the (x, y, z) components to RGB channels: red for the X axis, green for the Y axis and blue for the Z axis. This allows surface orientations to be visualized with colors, although these do not have a real chromatic meaning but rather a geometric one.

4.1.3 Implicit Functions

An **implicit function** in 3D is a mathematical function $f(x, y, z)$ that defines a surface or volume continuously in space. The surface is defined by the set of points where the function vanishes, that is: $f(x, y, z) = 0$. For example, a sphere of radius r centered at $(0, 0, 0)$ can be implicitly defined as $f(x, y, z) = x^2 + y^2 + z^2 - r^2 = 0$. Unlike explicit representations such as meshes, an implicit function does not directly store the geometry, but rather describes its shape using a continuous function.

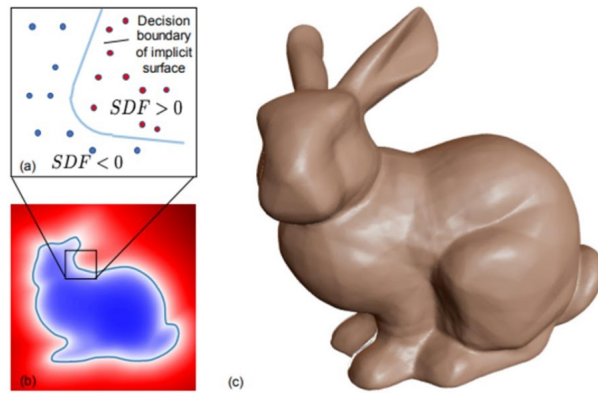


Figure 4.3: Implicit function for a rabbit. Note that in the surface the function is zero, outside greater than zero, and inside smaller than zero.

Note

We can interpret $f(x, y, z)$ like the “**distance to the surface**” of the volume. Therefore, in a point outside the volume we have that $f(x, y, z) > 0$; in a point inside the volume we have that $f(x, y, z) < 0$; and in a point exactly in the surface of the volume we have that $f(x, y, z) = 0$. This may not always be the Euclidean distance, as for example in the sphere the distance would be $\sqrt{x^2 + y^2 + z^2} - r$ instead of $x^2 + y^2 + z^2 - r^2$.

Warning: Implicit Functions

The main advantage is the continuous and infinite representation. It is not limited to a discrete grid, as you can evaluate any point. However, it is expensive to evaluate the function to reconstruct dense geometry if we want to evaluate all points of a zone, so it requires specialized sampling and optimization techniques.

4.1.4 Voxels

A **voxel** (volumetric pixel or volume element) is the basic unit in a regular three-dimensional representation, analogous to a pixel in 2D. While a pixel represents a point (small square) in a flat image, a voxel represents a point (**small cube**) in 3D space.

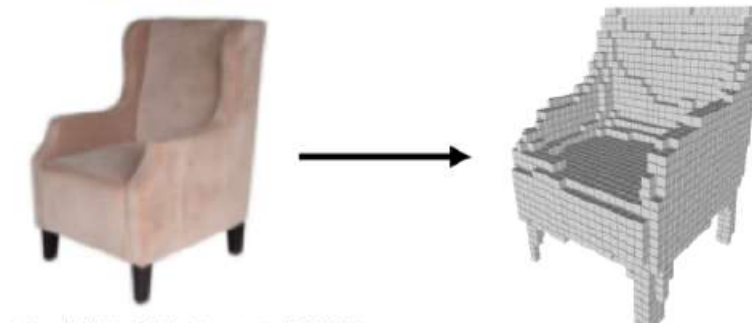


Figure 4.4: Example of a chair and its voxel.

A voxel-based structure divides a three-dimensional volume into a regular grid of cubes, and each voxel can store information such as occupancy (is there matter in that cube?); color or intensity; or physical properties (density, temperature, etc.). Voxels are useful for representing 3D geometry. Usually they do not store information, only if there is a cube or not in that position, so its shape is $V \times V \times V$, but if they store some information, for example density and temperature at each point, the shape would be $V \times V \times V \times 3$ because we have three features: there is a cube or not, density and temperature.

Warning: Voxels

Voxels a regular and direct representation of space. However, they are spatial inefficient: occupying an entire regular grid is very memory-intensive, especially at high resolution.

4.1.5 Point Cloud

A **point cloud** is a representation of a 3D shape or scene as a **set of points** in space. Each point has (x, y, z) coordinates that define its position in a three-dimensional reference system, and sometimes includes additional attributes such as color (RGB), intensity, or surface normals.

Unlike a regular representation such as voxels, a point cloud is a **sparse** representation: it only stores information at the locations where data actually exists, making it efficient for describing 3D geometry.

Warning: Point cloud

Point clouds are lightweight and efficient because they only store relevant data. They are also flexible as they do not impose a fixed grid like voxels. However, they are unstructured (points have no explicit connectivity) and it is difficult to use in standard convolutional networks, as there is no regular grid topology.



Figure 4.5: Two different views for the reconstructed point cloud of an airplane.

4.2 3D Convolutions

4.2.1 Model

A **3D convolution** extends the standard 2D convolution by operating over three spatial or spatiotemporal dimensions. Following the PyTorch convention, an unbatched input has shape

$$(C_{\text{in}}, D, H, W).$$

For video, the “depth” dimension D is usually denoted by T (“time”), so an unbatched input of videos has shape (C_{in}, T, H, W) .

3D convolutions learn features that combine spatial and temporal or volumetric information. However, they have high computational and memory costs because their kernels and activations extend across three dimensions.

A 3D convolutional layer with C_{out} filters has a weight tensor of shape

$$(C_{\text{out}}, C_{\text{in}}, K_D, K_H, K_W).$$

The kernel slides along the depth, height, and width dimensions, but not along the channel dimension. Each output channel combines information from all connected input channels.

The output spatial dimensions are

$$D_{\text{out}} = \left\lfloor \frac{D + 2P_D - \delta_D(K_D - 1) - 1}{S_D} + 1 \right\rfloor,$$

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_H - \delta_H(K_H - 1) - 1}{S_H} + 1 \right\rfloor,$$

and

$$W_{\text{out}} = \left\lfloor \frac{W + 2P_W - \delta_W(K_W - 1) - 1}{S_W} + 1 \right\rfloor,$$

where the stride, padding, and dilation are respectively

$$S = (S_D, S_H, S_W), \quad P = (P_D, P_H, P_W), \quad \delta = (\delta_D, \delta_H, \delta_W).$$

and they work as in 2D convolution but also act along the depth or temporal dimension.

Therefore, the complete output shape is

$$(C_{\text{out}}, D_{\text{out}}, H_{\text{out}}, W_{\text{out}}).$$

For video, this becomes

$$(C_{\text{out}}, T_{\text{out}}, H_{\text{out}}, W_{\text{out}}).$$

Stride, padding and **dilation** work the same as in 2D convolutions, but now they also include the depth dimension.

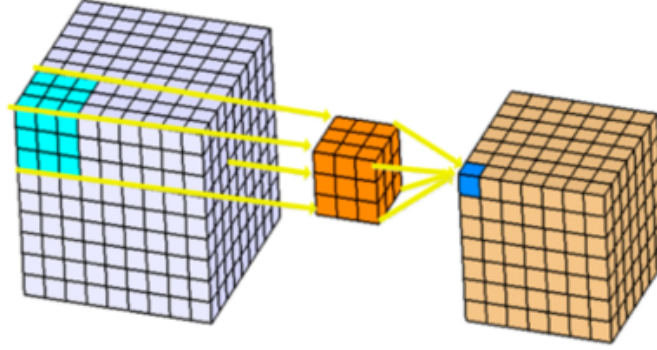


Figure 4.6: Example of a 3D convolution over one channel. The kernel moves along the depth, height, and width dimensions, while each output channel combines information from all connected input channels.

4.2.2 Max Pooling (3D) and Average Pooling (3D)

Max Pooling 3D takes a block of shape (P_D, P_H, P_W) as input and returns the maximum value of that block. **Average Pooling 3D** works the same, but returns the mean value of that block.

The output shape of these operations are the same as in the 3D convolution. Note that the max pooling is like if we had a kernel with zeros in all positions, except where the maximum value lives, where there would be a one; and in the average pooling is like if we had a $1/(K_D K_H K_W)$ in all positions.

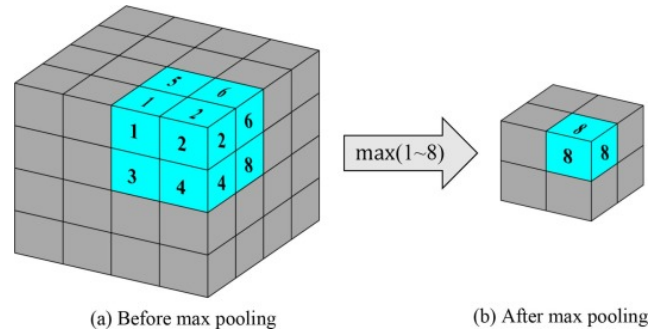


Figure 4.7: Max Pooling 3D. Note that the average pooling would be the same, but taking the mean of the block instead of the maximum.

4.3 Predicting Representations

4.3.1 Predicting Depth Maps

One straight way to train a model to predict depth maps is to train a fully convolutional network, as in the previous chapter, but now using only one output channel, as we only have to predict one value (depth) per pixel. We can use any metric that works for regression for the loss function, for example L_2 distance (mean squared error). This process is shown in Figure 4.8.

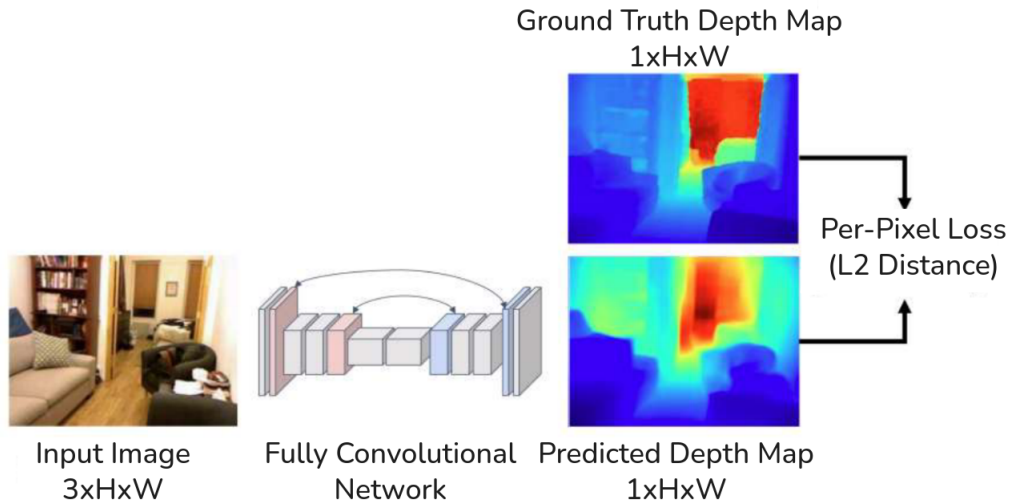


Figure 4.8: Schema of the training for predicting depth maps.

However, small, close objects look exactly the same as larger, farther away objects, so absolute scale or depth is ambiguous for a single image. You can see this with the example of Figure 4.9. Therefore, we can improve the previous approach.

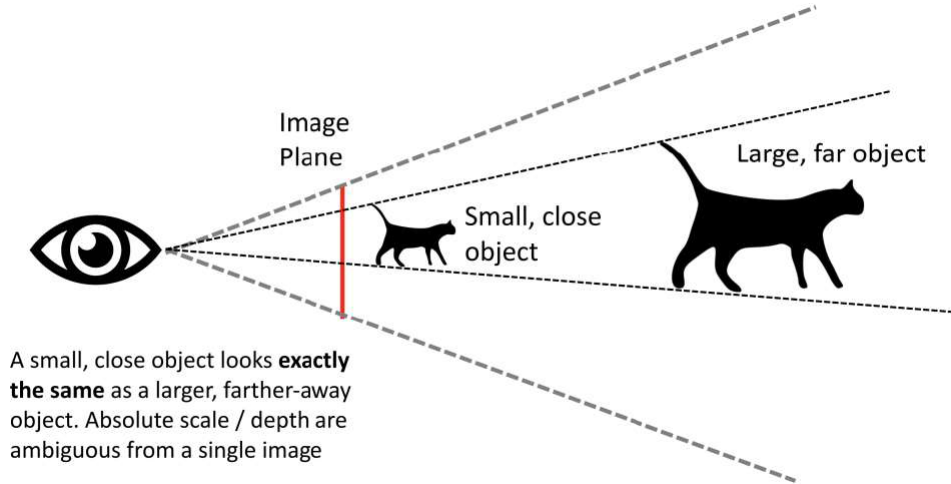


Figure 4.9: Problem: scale/depth ambiguity.

The **scale-invariant loss** is a loss function designed to make the model insensitive to absolute scales in the data. It is widely used, for example, in tasks such as depth estimation or 3D reconstruction, where the important thing is the relative structure (the relationships between points) and not the absolute magnitude of the distances. A typical way of defining it is:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W (\log(y_{ij}) - \log(\hat{y}_{ij}))^2 - \frac{\lambda}{(HW)^2} \left(\sum_{i=1}^H \sum_{j=1}^W (\log(y_{ij}) - \log(\hat{y}_{ij})) \right)^2,$$

where y_{ij} and $\hat{y}_{ij}$ are the ground truth and prediction for pixel (i, j) and $0 \leq \lambda \leq 1$ is a parameter typically set to one.

Note

The intuition behind this loss is that the first part measures the mean squared logarithmic error and the second part compensates for constant shifts in scale. If $\lambda = 0$, it totally depends on the scale; if $0 < \lambda < 1$, we obtain a compromise; and if $\lambda = 1$ we obtain a total scale-invariant loss. This means that when $\lambda = 1$, if you multiply all predictions by a constant factor, i.e., $\hat{y}'_{ij} = c\hat{y}_{ij} \forall i = 1, \dots, H; \forall j = 1, \dots, W; c > 0$, the loss does not change: $\mathcal{L}(y, \hat{y}) = \mathcal{L}(y, c\hat{y})$. Note that if we do not use the logarithm in both terms, this property would not hold.

4.3.2 Predicting Surface Normals

The approach to predicting surface normals is very similar to the one predicting depth maps. In fact, the process is all the same except that the output channels of the fully convolutional network are three (for the normal vector in each pixel) and the loss function used. Now we

use as loss function the **cosine distance**:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \underbrace{\left(1 - \frac{\vec{y}_{ij} \cdot \vec{\hat{y}}_{ij}}{\|\vec{y}_{ij}\|_2 \|\vec{\hat{y}}_{ij}\|_2} \right)}_{\text{cosine distance}},$$

where $\vec{y}_{ij} = (y_{ij}^1, y_{ij}^2, y_{ij}^3)$ and $\vec{\hat{y}}_{ij} = (\hat{y}_{ij}^1, \hat{y}_{ij}^2, \hat{y}_{ij}^3)$ are the ground truth and predicted surface normals. Recall that we want the ground truth and predicted vectors to be similar, and the smaller the cosine distance, the closer these vectors are, as shown in Figure 4.10. More precisely this distance is in the range $[0, 2]$. If the vectors point in the same direction, the cosine distance is 0; if the vectors are orthogonal, the cosine distance is 1; and if the vectors are perfectly opposite, the cosine distance is 2. The rest of the cases are between these values.

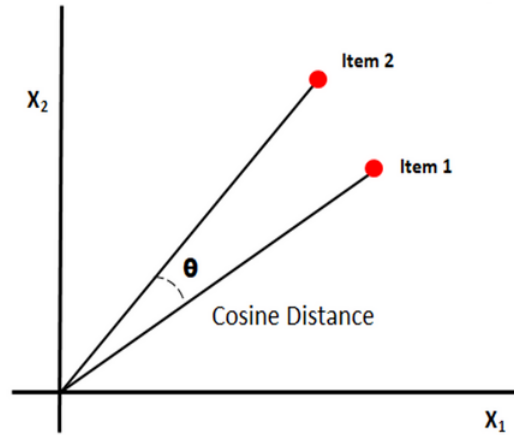


Figure 4.10: Cosine distance between two vectors. In this case, it will be near to zero.

4.3.3 Simulating Implicit Functions

To train a model that can simulate an implicit function, we can sample some random points of the space and generate the labels we want for that point (for example, their distance to the surface). Then, we can train any model with the inputs (points sampled) and labels generated. Note that the inputs usually will have dimension 3 and the output dimension 1.

4.3.4 Classifying Voxelized Objects

If we want to predict to what class a voxel belongs to, we can apply a 3D convolutional neural network, flatten the resulting block and then use a multilayer perceptron (MLP) to obtain the class. The loss function will be cross entropy for multiclass and binary cross entropy for binary classification. An example of an architecture for this problem is shown in Figure 4.11.

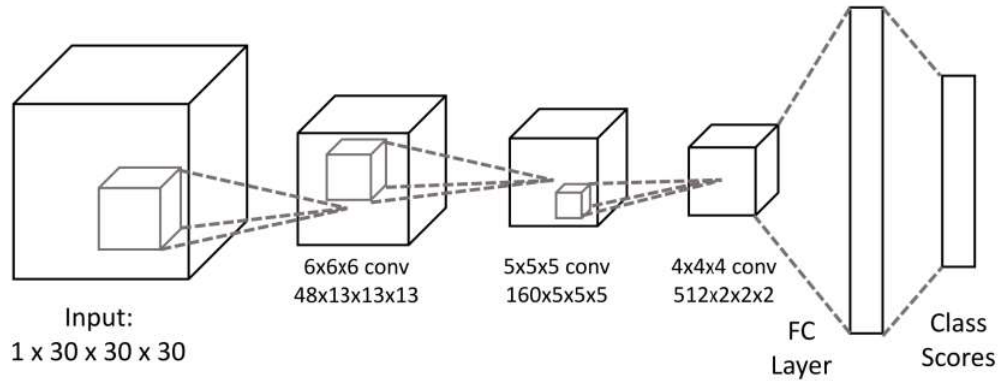


Figure 4.11: Architecture to predict what class a voxel belongs to. Note that we could also add 3D max pooling or average pooling between the 3D convolutions.

4.3.5 Predicting Point Clouds

To predict to what class a point cloud belongs to, we can pass each point through a MLP, aggregate all these output vectors with a **permutation invariant function**¹ applied over the point dimension, for example mean or maximum, and finally pass the obtained vector through another MLP to obtain the logits for each class. The loss function will be cross entropy for multiclass and binary cross entropy for binary classification. An example of an architecture for this problem is shown in Figure 4.12.

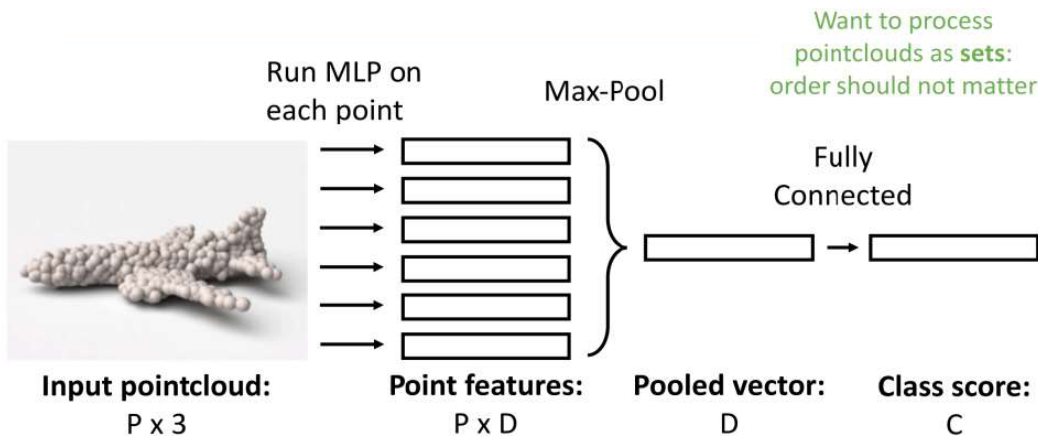


Figure 4.12: Example of architecture to predict what class a point cloud belongs to.

¹A permutation invariant function is a function that for every possible permutation of the input, it returns the same value. The mean would be an example: $f(1, 2, 3) = (1 + 2 + 3)/3 = 2 = f(1, 3, 2) = f(2, 1, 3) = f(2, 3, 1) = f(3, 1, 2) = f(3, 2, 1)$.

4.4 Predicting Shapes

4.4.1 Metrics

Intersection over Union (IoU)

This metric is the same seen for object detection. Just to recall, the IoU for two sets is

$$d_{\text{IoU}}(P, Q) = \frac{|P \cap Q|}{|P \cup Q|}.$$

Warning: IoU

In 3D shapes this metric is not the best one. It can not capture thin structures, it can only be applied to voxels and, in general, low values are not very meaningful. The explanation about why it can only be applied to voxels is simple: voxels can only take a finite number of values. However, in other representations, like point clouds, the points can take continuous values. Therefore, the intersection between two sets of points will be almost always zero.

Chamfer Distance

The Chamfer distance is a widely used metric for comparing two point clouds (P and Q) in 3D because it does not depend on exact correspondences between points. The mathematical formula is:

$$d_C(P, Q) = \frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \{\|p - q\|_2^2\} + \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \{\|q - p\|_2^2\}.$$

The intuition here is:

1. For each point $p \in P$, find the closest point in Q and measure the squared euclidean distance.
2. Then do the same for each point $q \in Q$, finding its closest point in P .
3. Add the two means.

In other words: it measures how well P covers Q and vice versa.

In addition, it has the following properties:

- It does not need to know which point in P corresponds to which in Q .
- The Chamfer distance is 0 for equal sets, i.e., if $P = Q$ then $d_C(P, Q) = 0$.
- It is symmetric: $d_C(P, Q) = d_C(Q, P)$.

- It is differentiable, so you can use it as a loss function to train neural networks.
- It can be applied to any shape representation.

Warning: Chamfer distance

In some cases it can be sensitive to outliers.

F1 Score

In the context of 3D shapes (point clouds, meshes, voxels, etc.), F1 score is often used to evaluate the comparison between two 3D shapes. Let's say the set of predicted points is P and the set of ground truth points is G . Then:

1. A distance threshold δ is set, for example 1% of the object size.
2. A point $p \in P$ is true positive (TP) if it is within a distance less or equal than δ of a G point. Otherwise, it is a false positive (FP).
3. A point $g \in G$ that is not covered by any P point is considered as false negative (FN).
4. We define precision as the percentage of predicted points that are close to the actual shape:

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

5. We define recall as the percentage of actual shape points that were covered by the prediction:

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

6. Finally, we define:

$$\text{F1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

It can be applied to any shape representation. It is believed it is the best shape prediction metric in common use.

Warning: F1 Score

It is sensitive to the threshold chosen.

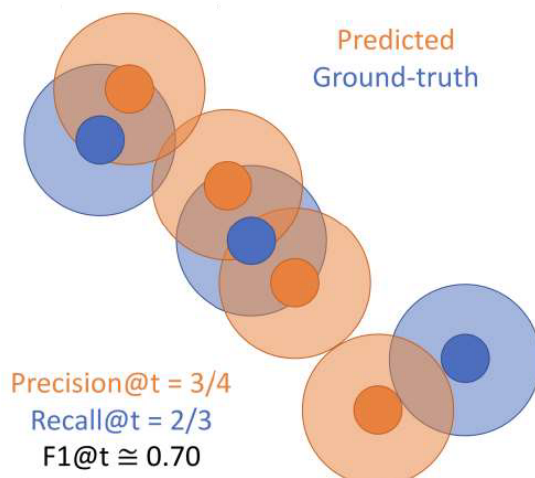


Figure 4.13: F1 score for a predicted set of points and its ground truth set of points (“@t” refers to the threshold used).

4.4.2 3D Datasets

ShapeNet is one of the most widely used databases in the field of computer vision and 3D modeling. It contains more than 50k 3D models hierarchically organized into 55 categories, such as chairs, airplanes, and cars. These models are represented as polygonal meshes and have been carefully curated and aligned to facilitate tasks such as shape reconstruction, classification, and segmentation. ShapeNet also includes variants such as ShapeNetCore, which offers a cleaner and more consistent subset of objects, and ShapeNetSem, which provides detailed semantic annotations for each model. Thanks to its diversity and size, it has become a standard for training and evaluating neural networks in tasks related to understanding three-dimensional geometry.

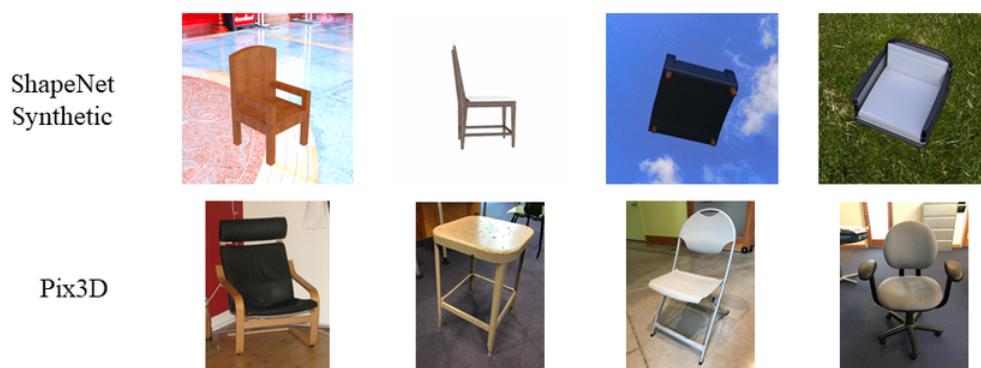


Figure 4.14: Some samples of the ShapeNet and Pix3D datasets.

Pix3D, on the other hand, is specifically designed to study the relationship between 2D images and 3D models. This dataset offers more than 10k real-world images of objects with their corresponding precisely aligned 3D models and associated pose (camera) data. Each instance includes mask and keypoint annotations, enabling tasks such as pose estimation, 3D

shape retrieval from images, and novel view generation. Unlike ShapeNet, which focuses on synthetic data, Pix3D incorporates real-world complexity, such as occlusion, variable lighting, and visual noise, making it an essential resource for validating algorithms that must operate under non-ideal conditions.

4.4.3 Generating Point Clouds

The objective in this section is to obtain the point cloud of a 2D image. For that, we first use a CNN to extract features and reduce the size of the original image. Then we use two parallel branches:

- The first branch uses a fully connected network to predict P_1 points (each of these points has 3 elements (x, y, z)). The idea here is to capture the global shape of the object.
- The second branch uses a CNN to obtain local points for each spatial position of the feature map extracted, concretely P_2 points per pixel. The idea here is to capture fine details and local variations. Note that the output of the CNN has to be reshaped from $3P_2 \times H'' \times W''$ to $H''W''P_2 \times 3$.

The points from both branches are concatenated to obtain a tensor with shape $(P_1 + H''W''P_2) \times 3$, that is, $P_1 + H''W''P_2$ points. This results in the complete point cloud, which combines the global structure and local details. Finally, the Chamfer distance of the ground truth set of points and the predicted set of points is used as loss function.

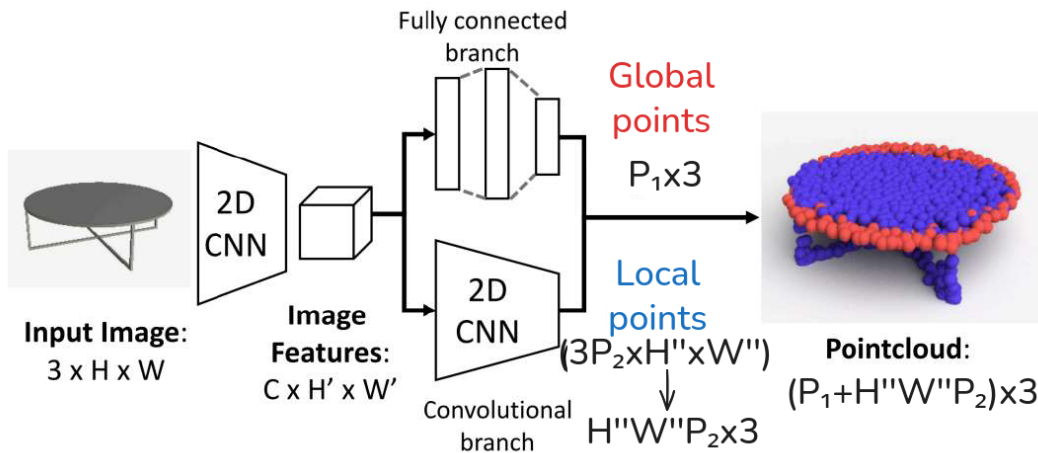


Figure 4.15: Generating a cloud point of a table. Red points represent the global shape and blue points represent local variations.

4.5 Video

4.5.1 Introduction to Video

In the context of computer vision, video analysis poses a number of additional challenges compared to processing still images. While an image can be represented as a two-dimensional array of pixels (height $\times$ width, with a possible third channel for color), a video introduces a temporal dimension, resulting in a three-dimensional (time $\times$ height $\times$ width) or even four-dimensional data volume if color channels are considered (time $\times$ channels $\times$ height $\times$ width). You can see this visually in Figure 4.16. This temporal dimension is critical because it contains information about motion, scene dynamics, and the evolution of objects over time.

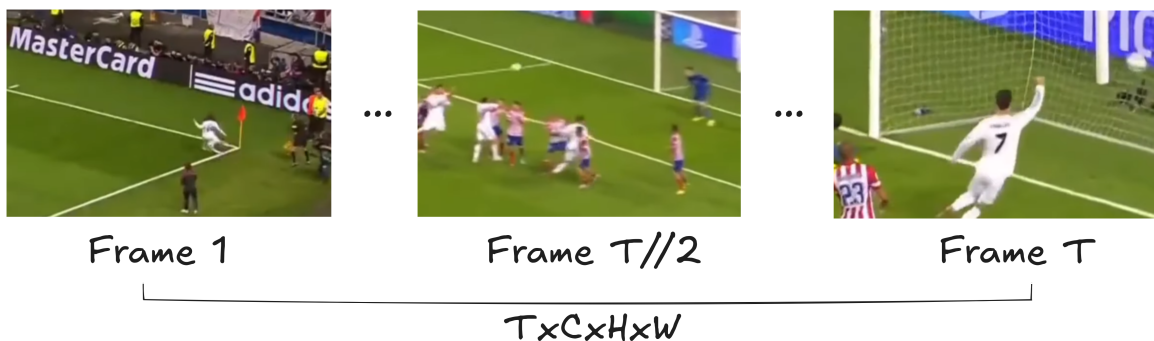


Figure 4.16: Videos incorporate a new dimension to images: the number of frames or time, usually denoted as T .

Working with video involves addressing specific problems such as lighting variability throughout sequences, motion blur, partial or total occlusion of objects, and the need to capture temporal dependencies at different scales. Computationally, videos are much heavier than images: each second of video at, for example, 30 fps, requires processing 30 images, which drastically increases storage, memory, and computing requirements.

From a deep learning perspective, this type of data requires models capable of capturing not only spatial information (patterns within each frame) but also temporal relationships between consecutive frames.

4.5.2 Action Recognition

Action recognition is one of the fundamental tasks in computer vision applied to video. It involves identifying and classifying actions or activities that occur in a temporal sequence, such as “walking”, “running”, “jumping”, or “clapping” (Figure 4.17). Unlike object recognition in static images, this task requires understanding not only the spatial appearance of individual frames but also the temporal evolution of movements.

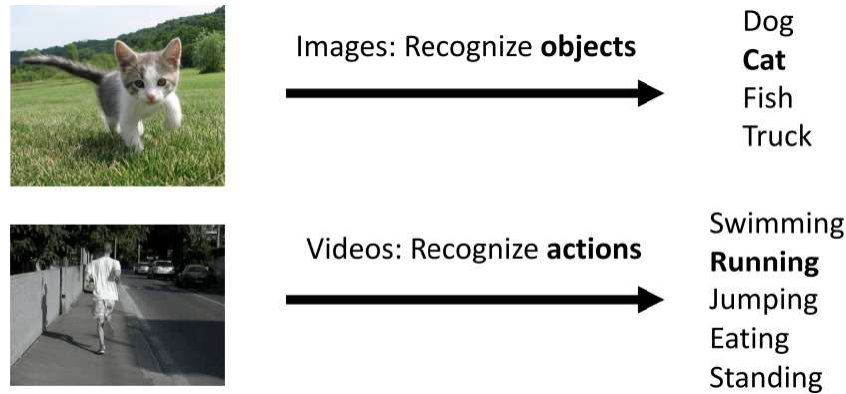


Figure 4.17: Difference between image classification and video classification.

One of the main difficulties in action recognition lies in the high intra-class variability: the same action can be executed in very different ways depending on the person, speed, camera angle, or lighting conditions. Furthermore, there are classes of actions with very subtle differences (e.g., “drinking” vs. “eating”) that require models capable of capturing fine details in space and time.

Action recognition has direct applications in fields such as intelligent surveillance, human-computer interaction, augmented reality, and sports analysis, making it an area of great interest both academically and industrially. In terms of deep learning, various approaches have been proposed to address this task, which we will see along this section.

Note

In many cases, as the videos are very heavy in memory (for example, for 30 fps HD 1920×1080 is ~ 11 GB per minute), the following strategy is followed: the video is resized to a smaller shape and also the fps are reduced.

Single-Frame CNN

A simple idea is to train a normal CNN to classify images. Then we will use this CNN to classify each frame (or a subset of all the frames) of the video. Finally, we can take the mean to obtain the final prediction. The layer where the predictions are aggregated is usually called fusion layer. You can see the workflow in Figure 4.18.

Although this technique may seem basic, it is a very good starting point when classifying videos and we can use it as a baseline to compare other methods.

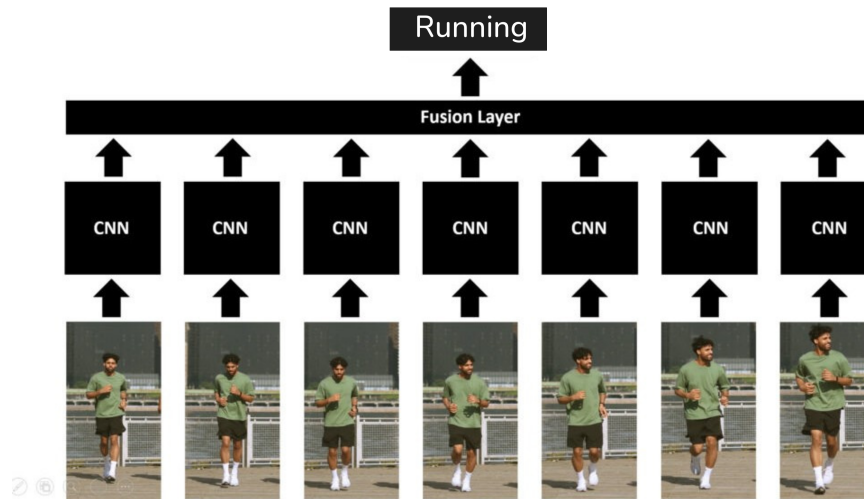


Figure 4.18: Workflow of the single-frame CNN.

Late Fusion

A more refined way to approach video classification consists on running a CNN for each frame to obtain their feature maps, then flatten these feature maps, and use a MLP to obtain the final class scores. This is called late fusion with FC layers.

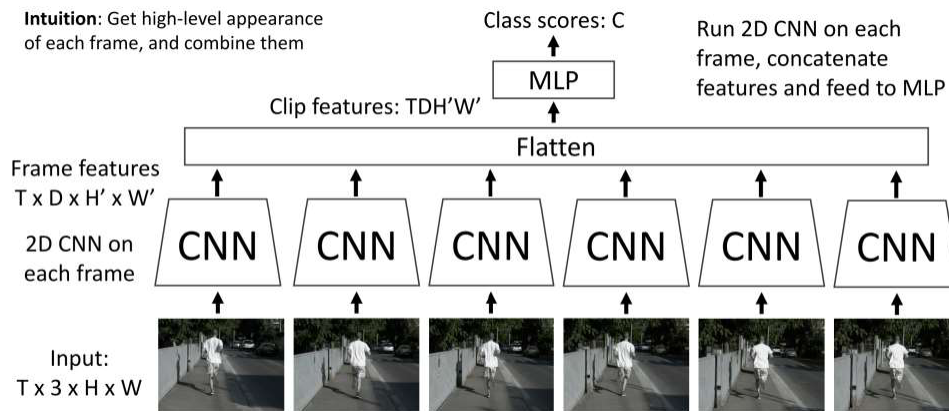


Figure 4.19: Schema of late fusion with FC layers.

Another variant of this method to avoid $TDH'W'$ being too large is late fusion with pooling, which also runs a CNN in each frame, obtains their feature maps, and instead of flatten and concatenate them, the mean is taken over the time (first dimension) and space (third and fourth dimensions). Finally, we use again a MLP to obtain the final class scores.

The main idea in late fusion is to get a high-level appearance of each frame and combine them. However, it is hard to compare low-level motion between frames. Note that it is called late fusion because the frames are merged after using the CNN.

Early Fusion

Early fusion tries to combine all frame in one tensor and use this as input for the CNN. That is, if the video has shape $T \times 3 \times H \times W$, we reshape it into a tensor of shape $3T \times H \times W$ and then we fed this into a 2D CNN. The architecture of the CNN here is very flexible: we can use as many layers as we want and include a MLP at the end to predict the logits for each class.

One problem here is that the number input channels may be too large for the CNN. Note that is called early fusion because the frames are merged before using the CNN.

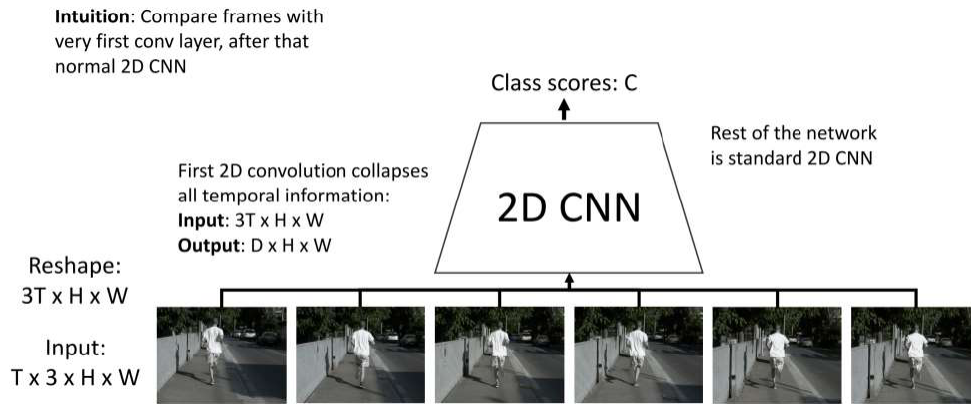


Figure 4.20: Schema of early fusion.

3D CNN

3D CNNs (seen previously in this chapter) apply convolutions to 4D tensors to fuse temporal information over the course of the network. Some 3D convolutional layers or 3D max/average pooling can be stacked. Finally, the feature maps are flattened and a MLP is used to classify the video.

Note that the process is very similar to early fusion, but using a 3D CNN instead of a 2D CNN, that is, the tensors have dimension $T \times C \times H \times W$ instead of $CT \times H \times W$.

Temporal shift invariance is a desirable property in models that process temporal sequences (such as video) and means that the model produces the same output even if the input is slightly shifted in time (for example, if the action occurs a little earlier or later).

- In the context of early fusion, data from different frames is combined before processing and it is not invariant to temporal shifts because the model sees the frames in a fixed order. If you change the order or shift the frames, the output changes.
- 3D CNNs apply convolutions in space and time. If well designed (for example, with 3D kernels and without excessive temporal padding), they can capture local temporal patterns and be more robust to small shifts in time. However, they are not perfectly invariant, only tolerant of certain small shifts.

Layer	Size
Input	3 x 16 x 112 x 112
Conv1 (3x3x3)	64 x 16 x 112 x 112
Pool1 (1x2x2)	64 x 16 x 56 x 56
Conv2 (3x3x3)	128 x 16 x 56 x 56
Pool2 (2x2x2)	128 x 8 x 28 x 28
Conv3a (3x3x3)	256 x 8 x 28 x 28
Conv3b (3x3x3)	256 x 8 x 28 x 28
Pool3 (2x2x2)	256 x 4 x 14 x 14
Conv4a (3x3x3)	512 x 4 x 14 x 14
Conv4b (3x3x3)	512 x 4 x 14 x 14
Pool4 (2x2x2)	512 x 2 x 7 x 7
Conv5a (3x3x3)	512 x 2 x 7 x 7
Conv5b (3x3x3)	512 x 2 x 7 x 7
Pool5	512 x 1 x 3 x 3
FC6	4096
FC7	4096
FC8	C

Figure 4.21: Architecture of C3D.

One of the most famous architectures is **C3D**, known also as the VGG of 3D CNNs. Its architecture is shown in Figure 4.21. The main problem is that it is very expensive computationally. Just to have some numbers to compare, AlexNet has 0.7 GFLOP, VGG-16 13.6 and C3D 39.5 GFLOP ($\sim 3\times$ VGG)!

Note

The single-frame CNN and 3D CNNs can handle videos of arbitrary length, while the videos for early fusion and late fusion must always have the same number of frames.

Two-Stream Flow Networks

Optical flow is a technique that calculates the apparent motion of pixels between two consecutive frames in a video. It represents motion vectors: how much and in what direction each pixel moves. The result is typically an image where each pixel has a direction and magnitude of motion. It captures temporal information (motion), which a normal 2D network (which sees static images) cannot directly learn. An example is shown in Figure 4.22.

There are some techniques to calculate optical flow, but they will not be discussed here. Some examples are TV-L1 or RAFT, and some libraries have them already implemented, like `cv2`.

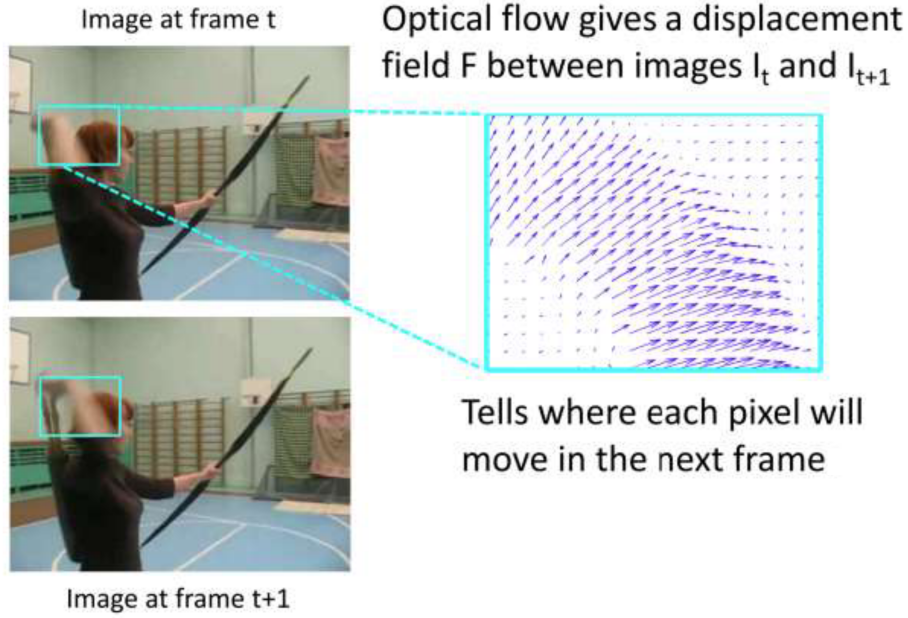


Figure 4.22: Example of optical flow. Note that $F_t(x, y) = (dx_t, dy_t)$ and $I_{t+1}(x, y) = I_t(x, y) + F_t(x, y) = (x_t + dx_t, y_t + dy_t)$.

Two-Stream network is an architecture proposed for action recognition in video. It has two branches (streams):

- Spatial stream (RGB): Takes normal RGB images (appearance) and learns what is seen (people, objects).
- Temporal stream (optical flow): Takes optical flow maps (motion between frames) and learns how things move.

Both networks are CNNs (usually 2D), and are combined at the end to make the final prediction. They explicitly combine appearance and motion, usually outperform RGB-only CNNs in action tasks and require less data than 3D models like C3D, because it is based on pre-trained architectures (such as VGG or ResNet). However, optical flow is expensive to compute (offline or with a separate model), double memory is used: two complete networks (more weight and computation) and they may be sensitive to errors in optical flow.

There are many ways to pass the inputs. Note that if we have T frames, as for computing the optical flow between two images we need that two images, we can obtain at most $T - 1$ optical flows (1 and 2, 2 and 3, $\dots$, $T - 1$ and T).

- For the spatial stream, we can use any architecture seen before (single-frame CNN, late fusion, early fusion or 3D CNN). Another approach to reduce the computation is to just pass one representative frame of the video.
- For the temporal stream, the optical flows obtained are concatenated in the channels dimension (therefore, we obtain channels $dx_1, dy_1, \dots, dx_{T-1}, dy_{T-1}$) and the input dimensions for the CNN are $2(T - 1) \times H \times W$.

At the end, we can concatenate the feature maps obtained in both networks and use a MLP to obtain the final prediction or predict in each network and then average the results. Other option, in order to reduce the memory used, is to just use one CNN for the spatial and temporal streams; however, in this case, we can not use 3D CNN. A possible workflow of the model is shown in Figure 4.23. Note that there are more possible options with this type of models. Not only those we have discussed in these notes, but also more in the literature.

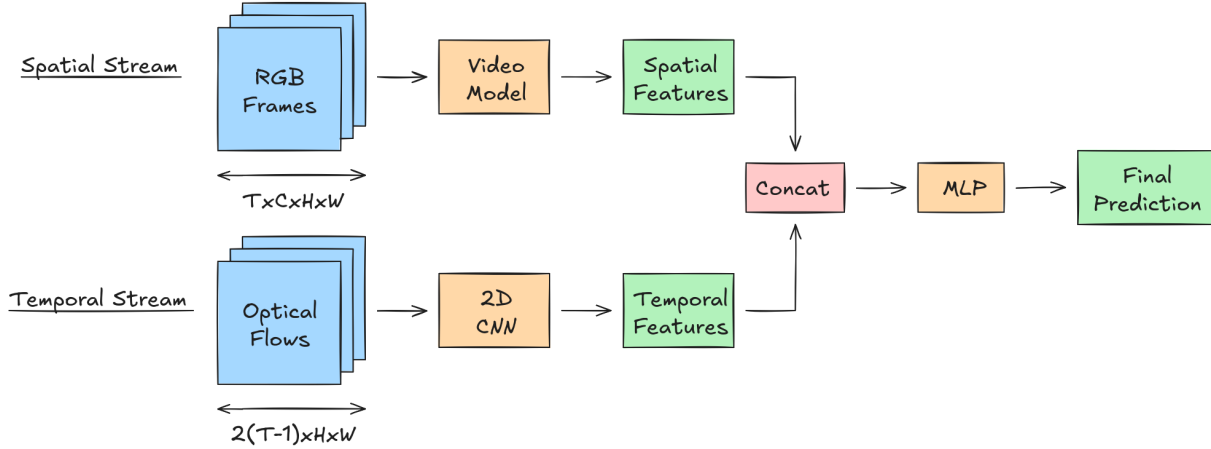


Figure 4.23: Possible workflow for two-stream flow networks.

Inflated CNNs

Inflated 3D ConvNets (I3D) adapt pretrained 2D CNNs to video by adding a temporal dimension to their filters. A 2D convolution weight has shape

$$W_{2D} \in \mathbb{R}^{C_{out} \times C_{in} \times K_H \times K_W},$$

and its inflated 3D weight has shape

$$W_{3D} \in \mathbb{R}^{C_{out} \times C_{in} \times K_T \times K_H \times K_W}.$$

The weights are initialized as

$$W_{3D}[:, :, t, :, :] = \frac{1}{K_T} W_{2D}, \quad t = 0, \dots, K_T - 1.$$

This way, a “static video” (a video where every single frame is the exact same image), will output the exact same result as the original 2D network.

If all temporal slices are identical, their K_T responses sum to the original 2D response. Without the factor $1/K_T$, the initial preactivation would be multiplied by K_T . This is only an initialization: the temporal slices can learn different weights during training.

4.5.3 Comparison of the Models

The **Sports-1M** dataset, developed by Google Research in 2014, is one of the pioneering and most influential datasets for video analysis in the context of sports activity recognition.

It consists of approximately 1.1 million videos automatically extracted from YouTube, which were classified into approximately 487 different sports-related categories. Unlike other manually annotated datasets, the labels in Sports-1M were automatically generated from the metadata, titles, and descriptions associated with the videos, which facilitates large-scale collection but introduces a considerable level of noise into the labels.

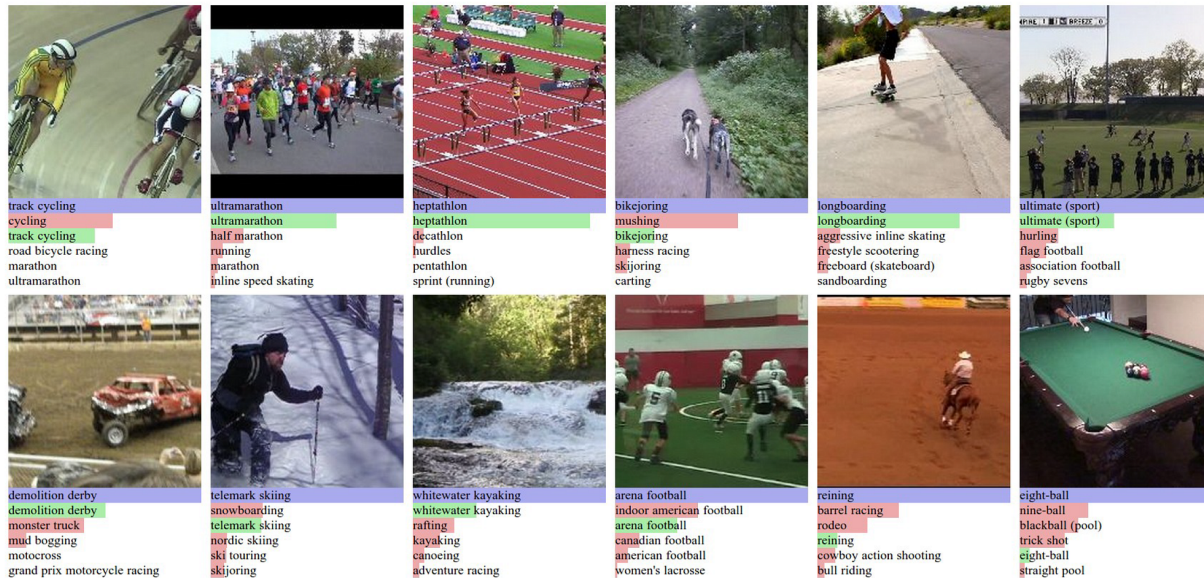


Figure 4.24: Some examples of the Sports-1M dataset.

The videos vary widely in length, resolution, and visual quality, reflecting the diversity of online content. On average, each video is about five minutes long and can be labeled with multiple classes, making it a multi-class, multi-label classification dataset. This variability and volume make Sports-1M an ideal resource for testing deep learning models, especially those seeking to capture complex temporal patterns, such as 3D convolutional networks or LSTM-like recurrent architectures.

One of the highlights of Sports-1M is its relevance for video action and activity classification tasks, although its practical use presents certain challenges. Access to the dataset is not direct: due to licensing restrictions imposed by YouTube, the dataset does not include the videos themselves, but only a list of IDs that the user must manually download, assuming the videos are still available on the platform. Furthermore, automatic labeling can lead to incorrect or ambiguous classifications, so many researchers choose to perform additional filtering or validation before using the data in supervised tasks.

Some examples of the dataset are shown in Figure 4.24. Also, the models seen in the previous section have been tested in this dataset. You can see these comparisons in Figure 4.25, where the metric used is top-5 accuracy².

²Top-5 accuracy is a metric primarily used in multiclass classification, especially in tasks with a large number of classes, to assess whether the model has included the correct class among its five most likely predictions. Rather than requiring the model to accurately predict the most likely class (as in top-1 accuracy), top-5 accuracy allows a prediction to be considered correct if the true class is among the five most likely classes assigned by the model.

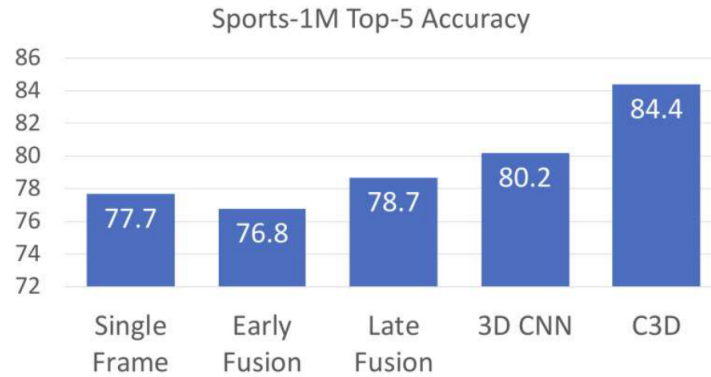


Figure 4.25: Top-5 accuracy on Sports-1M for different models.

4.5.4 Long-Term Temporal Structure

So far, all our temporal CNNs only model local motion between frames in short clips. What about long-term structure?

LSTMs for Video

A natural way of modelling sequences is using recurrent networks, specially Long Short-Term Memory networks (LSTMs). To apply LSTMs to video, you can treat each video as a sequence of frames (images). First, you extract features from each frame using a convolutional network (CNN), such as ResNet or VGG. Then, those features (vectors that summarize the visual content of each image) are passed sequentially to an LSTM, which learns the temporal dependencies between frames (e.g., motion or an action that develops over time). Finally, the output of the LSTM is used in a fully connected layer to obtain the final prediction. Note that the main idea here is to extract local information with the CNNs and then obtain a global view of the video with the LSTM.

The CNN used can be trainable, but usually its weights are fixed to make training faster and more efficient. Also, the output can be one prediction per frame or just one final prediction, as in Figure 4.26. This depends on the task we are doing. For example, for video classification only one output is needed, but for video segmentation or if we know more than one action will be done and we want to classify all actions done in the video we probably want one output per frame.

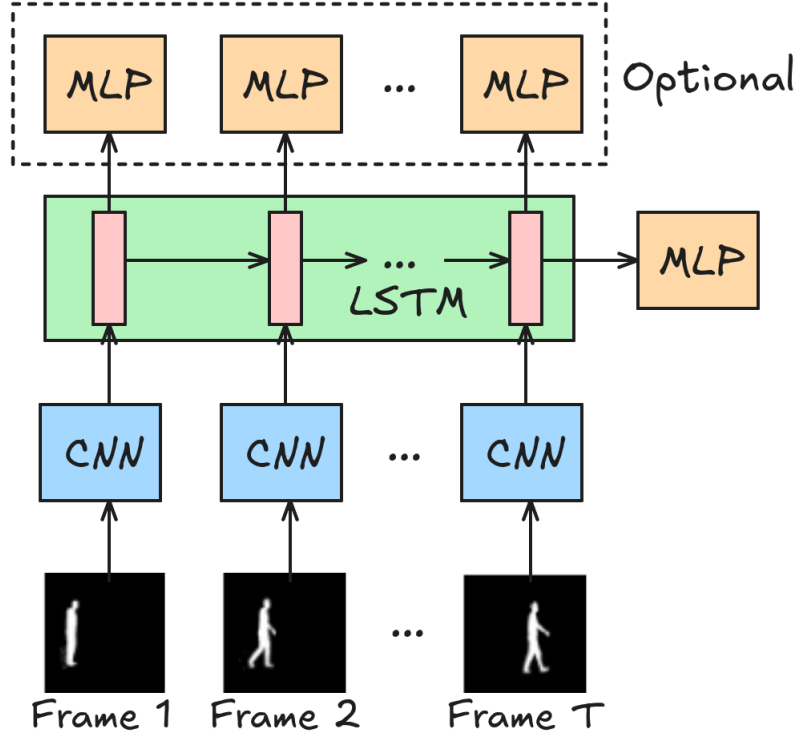


Figure 4.26: Using a LSTM for video.

SlowFast Networks

SlowFast networks process video using two parallel pathways:

- The **Slow pathway** uses a lower temporal sampling rate and learns spatial and semantic information.
- The **Fast pathway** uses a higher temporal sampling rate but fewer channels, focusing on motion and rapid changes.

If a Slow feature tensor has shape (C, T, H, W) , the corresponding Fast tensor has shape

$$(\beta C, \alpha T, H, W).$$

The two key ratios are:

- α is the ratio of the Fast to Slow temporal sampling rates; a typical value is $\alpha = 8$.
- β is the ratio of Fast to Slow channel capacity, so $C_{\text{Fast}} = \beta C_{\text{Slow}}$; typically $\beta = 1/8$, while $1/6$ has also been studied.

Lateral connections fuse Fast features into the Slow pathway at multiple network stages, combining detailed motion with richer spatial information.

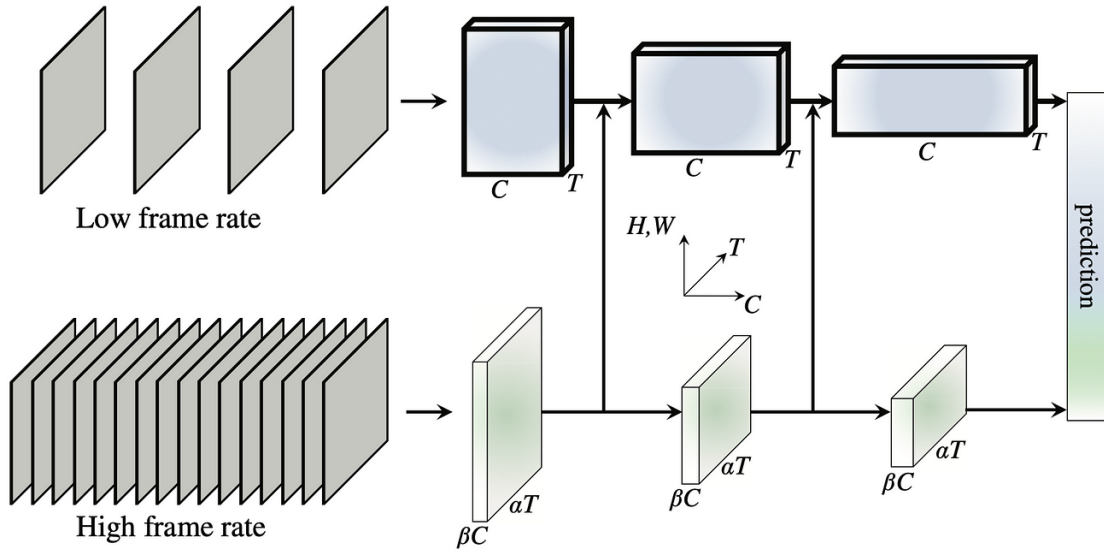


Figure 4.27: Architecture of a SlowFast network.

4.5.5 Other Problems in Video

During the chapter we have mainly focused on video classification because it is the most common task. However, there are some other problems that are worth mentioning:

- **Temporal Action Localization (TAL):** The goal is to detect when an action occurs in a long, unsegmented video. The input is a continuous video (e.g., several minutes) and the output are time intervals [start, end] along with the action label. For example, in a video of a person training, detect that a “jump rope” occurs between seconds 15 and 22, and a “push-up” occurs between seconds 30 and 40. Some challenges are that actions can be brief and subtle and that there may be multiple actions in succession or overlapping.
- **Spatio-Temporal Action Detection:** The goal is to detect when and where an action occurs in the video, that is, locate it in time and space (image). The input is a video and the output a bounding box around the actor and the action label for each frame. For example, to detect that a person is “kicking a ball” between seconds 10 and 13, and in each frame in that range, draw a box around the person. The main challenges are the challenges of TAL plus the challenges of spatial detection.



Figure 4.28: Example of temporal action localization.

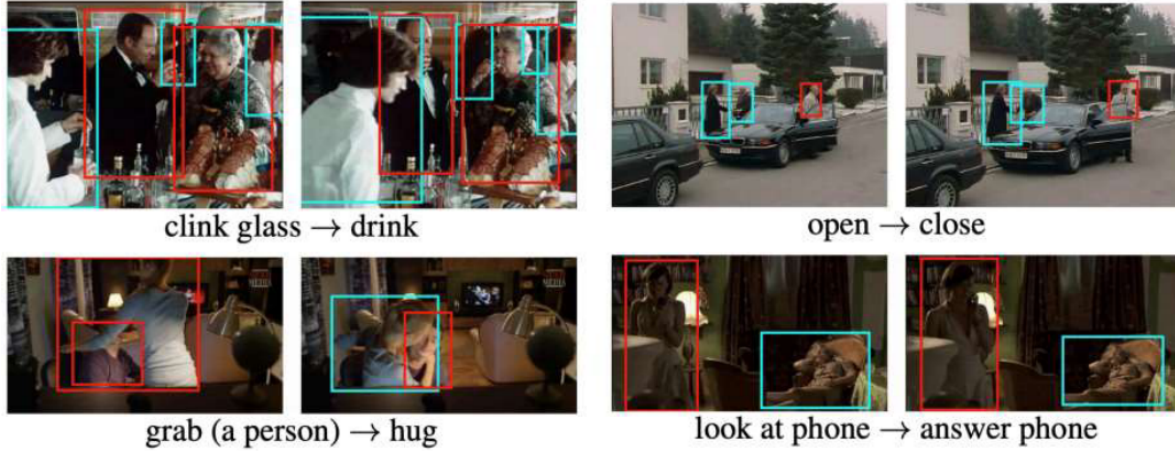


Figure 4.29: Examples of spatio-temporal action detection.

4.6 Exercises

Exercise 1: 3D Convolution Arithmetic

Given an input video tensor of shape $(T, C, H, W) = (16, 3, 112, 112)$ (representing 16 RGB frames of 112×112 pixels), calculate the exact output shape after applying a 3D convolutional layer with 64 filters of size $(K_T, K_H, K_W) = (3, 7, 7)$, a stride of $S = (1, 2, 2)$, no padding $P = (0, 0, 0)$, and normal dilation.

Exercise 2: Manual 3D Convolution

Perform a 3D convolution by hand on a single-channel input volume of shape $3 \times 1 \times 5 \times 5$. Define a simple $2 \times 2 \times 2$ kernel filled with small integer values (e.g., 1s, 0s and -1 s). Assume a stride of 1, padding of 0, and normal dilation. Show the step-by-step mathematical computation for the first spatial and temporal slice of the output tensor.

Exercise 3: Kernel Inflation

The Inflated 3D ConvNet (I3D) architecture adapts 2D image models to video by inflating 2D kernels to 3D. Given a pretrained 2D kernel $W_{2D} \in \mathbb{R}^{3 \times 3}$ and a target temporal dimension $T = 5$:

1. Write the mathematical formula to construct the inflated kernel $W_{3D} \in \mathbb{R}^{5 \times 3 \times 3}$.
2. Explain mathematically why dividing the weights by T ensures that the overall response magnitude of the filter remains consistent when applied to a static video (a video where all frames are identical).

Exercise 4: Scale-Invariant Loss

Prove mathematically that the scale-invariant loss function used for depth map estimation is perfectly invariant to global scale changes when $\lambda = 1$. Specifically, show that if all predictions are multiplied by a positive constant $c > 0$ (i.e., $\hat{y}'_{ij} = c\hat{y}_{ij}$), the loss remains unchanged: $\mathcal{L}(y, \hat{y}) = \mathcal{L}(y, \hat{y}')$.

Exercise 5: Surface Normal Distances

The cosine distance is used as a loss function for predicting surface normals. Calculate the cosine distance $\mathcal{L}(y, \hat{y})$ for a single pixel in the following three scenarios, assuming both $\vec{y}$ and $\vec{\hat{y}}$ are unit vectors:

1. The predicted normal is perfectly aligned with the ground truth.
2. The predicted normal is perfectly orthogonal to the ground truth.
3. The predicted normal points in the exact opposite direction of the ground truth.

Exercise 6: Memory Complexity (Voxels vs. Point Clouds)

Consider a 3D space of dimensions $100\text{m} \times 100\text{m} \times 100\text{m}$. We wish to represent a single hollow sphere of radius 10m located at the center.

1. If we use a voxel grid with a resolution of 0.1m per voxel, how many total voxels must be stored in memory?
2. If we use a point cloud that samples the surface of the sphere with a density of 100 points per square meter, approximately how many points must be stored? Discuss the efficiency trade-offs.

Exercise 7: Chamfer Distance

Given two 3D point clouds P and Q :

1. Prove that the Chamfer distance is symmetric: $d_C(P, Q) = d_C(Q, P)$.
2. Prove that if the sets are perfectly identical ($P = Q$), then $d_C(P, Q) = 0$.
3. Explain conceptually, or with a small visual example, why the Chamfer distance is highly sensitive to isolated outlier points.

Exercise 8: Point Cloud Metrics

Write a Python function to compute the F1 Score given two sets of 3D points (a predicted point cloud P and a ground truth point cloud G) and a distance threshold δ .

Exercise 9: Point Clouds

When classifying a point cloud, the architecture must handle unordered sets of points.

1. Mathematically demonstrate why a standard Multilayer Perceptron (MLP) applied directly to a flattened array of (x, y, z) coordinates fails to be permutation invariant.
2. Explain how applying a shared MLP to each point individually, followed by a global max-pooling operation across the point dimension, achieves strict permutation invariance.

Exercise 10: Video Architecture Baselines

Design and write the PyTorch pseudocode for three baseline video classification models given a video tensor of shape (B, T, C, H, W) :

1. Late Fusion with Pooling: Extract features using a 2D CNN per frame, then apply temporal average pooling before the final classifier.
2. Early Fusion: Reshape the input to (B, TC, H, W) and process it using a standard 2D CNN.
3. 3D CNN: Process the tensor directly using `torch.nn.Conv3d` and `torch.nn.MaxPool3d` layers.

Compare the parameter counts and the ability of each model to capture low-level temporal motion.